pc-dbCBS: Kinodynamic Motion Planning of Physically-Coupled Robot Teams

Khaled Wahba, *Member, IEEE*, and Wolfgang Hönig

***Abstract*—Motion planning problems for physically-coupled multi-robot systems in cluttered environments are challenging due to their high dimensionality. Existing methods combining sampling-based planners with trajectory optimization produce suboptimal results and lack theoretical guarantees. We propose Physically-coupled discontinuity-bounded Conflict-Based Search (pc-dbCBS), an anytime kinodynamic motion planner that extends discontinuity-bounded CBS to rigidly-coupled systems. Our approach proposes a tri-level conflict detection and resolution framework that includes the physical coupling between the robots. Moreover, pc-dbCBS alternates iteratively between state space representations, thereby preserving probabilistic completeness and asymptotic optimality while relying only on single-robot motion primitives. Across 25 simulated and six real-world problems involving multirotors carrying a cable-suspended payload and differential-drive robots linked by rigid rods, pc-dbCBS solves up to 90% more instances than a state-of-the-art baseline, planning trajectories up to 60% faster with significantly reduced planning time.**

***Index Terms*—Constrained motion planning, multi-robot systems, motion and path planning.**

I. Introduction

PHYSICALLY-COUPLED systems, such as multirotors collaboratively transporting cable-suspended payloads [1] or multiple mobile manipulators transporting objects [2], are increasingly used in real-world tasks requiring coordinated interaction. These systems are particularly valuable in environments such as construction sites for carrying tools or materials and in precision tasks requiring synchronized motion. The robot coupling introduces additional challenges, as the planned motions must respect both inter-robot dependencies and the system's dynamic constraints. There has been a significant focus on controlling such systems [3], [4] and only limited work on planning feasible motions in cluttered environments that require team formation changes. Moreover, existing planners produce motions that are rather slow and fail to exploit the agility of the underlying single-robot systems. This limitation arises from the reliance on a simplified model of the system, where the planner generates suboptimal, almost quasi-static plans [5].

In this work, we address the limitations of the current state-of-the-art [5], and present an anytime, probabilistically complete, motion planner for physically-coupled multi-robot systems in cluttered environments. To the best of our knowledge, this is the first work to offer this combination of simplicity, strong theoretical guarantees, and superior performance in the physically-coupled systems domain. Our approach builds on discontinuity-bounded Conflict-Based Search (db-CBS) [6], a multi-robot motion planning algorithm designed for uncoupled robots. Although db-CBS reasons over the stacked state space of the robots, it only accounts for collision avoidance and neglects other interactions. Directly enforcing physical coupling constraints during trajectory optimization is also impractical. The redundancy in the state space with the coupling constraints, results in numerical instabilities and ill-conditioned optimization, causing failure. On the other hand, pc-dbCBS introduces a tri-level conflict detection and resolution that (i) embeds the rigidity constraints between the robots and (ii) switches between state space representations while retaining all the completeness and optimality guarantees of db-CBS.

The main algorithmic contribution of this work is a general anytime, probabilistically complete, kinodynamic motion planning framework for **physically-coupled multi-robot systems**, relying only on single-robot motion primitives. Moreover, we provide a key idea of alternating between state space representations from the stacked to a minimal representation in the same planning framework while having completeness and asymptotic optimality guarantees. Empirically, we test our method in simulation and real experiments on two case studies: multirotors transporting payloads with cables and differential drive robots connected by rigid rods as shown in Fig. 1. We show that our method has higher success rate than the current state-of-the-art [5] and outperforms it in terms of cost, energy consumption, and computational time.

II. Related Work

Existing physically-coupled multi-robot systems include humanoid robots transporting payloads [7], multi-robot mobile manipulators moving objects [2], and aerial collaborative payload transport systems [5], [8], [9]. The problem of aerial manipulation with a single robot [10] and high-dimensional articulated robots [11], [12], [13] are also related, as they have to consider very large search spaces for motion planning.

Received 14 May 2025; accepted 20 August 2025. Date of publication 8 September 2025; date of current version 17 September 2025. This article was recommended for publication by Associate Editor T. Do and Editor A. Bera upon evaluation of the reviewers' comments. The work was supported by Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Grant 448549715. *(Corresponding author: Khaled Wabha.)*

The authors are with the Faculty of Electrical Engineering and Computer Science, Technical University of Berlin, 10623 Berlin, Germany (e-mail: k.wahba@tu-berlin.de; hoenig@tu-berlin.de).

This article has supplementary downloadable material available at https://doi.org/10.1109/LRA.2025.3607267, provided by the authors.

Digital Object Identifier 10.1109/LRA.2025.3607267

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

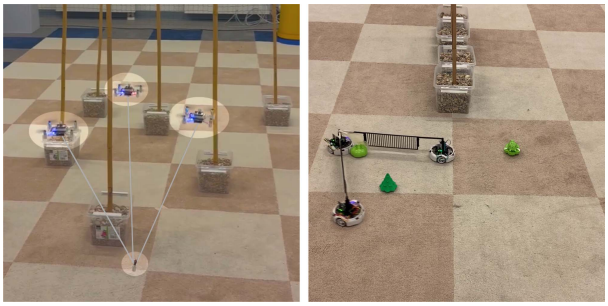

Fig. 1. Real experiments validation scenarios. Left: Three multirotors transporting a cable-suspended payload in a forest-like environment. Right: Three differential drive robots connected with rigid rods collecting items while avoiding obstacles.

In principle, such motion planning problems can be solved with any existing single-robot kinodynamic motion planner by stacking the states and actions. Common formulations use search-[13], sampling- [14], or optimization-based [15], [16] approaches. Special extensions for the multi-robot case can improve the scalability by leveraging the sparsity of the problem. These methods include sampling-based approaches such as K-CBS [17], optimization and control-inspired methods such as S2M2 [18], or hybrid methods such as db-CBS [6].

Motion planners for the physically-coupled case exist as well, especially for aerial transport of payloads. One approach extends a sampling-based method to the *Fly-Crane*, a system where three multirotors use two cables each to transport a rigid body [19]. In cases with few or no obstacles, a formation controller might also be used, rather than a full motion planner [20]. Other approaches rely on differential flatness [21] that provides a simplified framework for trajectory planning by leveraging flat outputs, but it typically leads to suboptimal solutions and is unable to directly account for motor actuation limits. Alternative methods that include the motor actuation limits combine ideas from sampling and optimization to compute feasible trajectories [5], [22]. In particular for cable-suspended payload transportation, [5] adopts this framework. Here, a simplified model is first used to warm-start a trajectory optimization step that incorporates necessary constraints. However, this method has two major limitations: i) the simplified model often produces suboptimal quasi-static hovering states, and ii) the lack of feedback between levels hinders completeness by limiting solution space exploration. One shortcoming of all existing methods is that the resulting motions are slow and do not exploit the full agile capabilities of the robots.

In contrast, this paper proposes an anytime planner that can produce time-optimal solutions, given enough computational effort. Our work is inspired by recent hybrid methods for uncoupled kinodynamic motion planning [6] and empirically compares to a hierarchical planner that was previously used for aerial transport [5].

III. Problem Formulation

We consider the motion planning problem of N robots that are physically-coupled with rigid connections, forming an actuated multi-robot system. The state of the system is represented as a stacked vector of each individual robot's state as follows

$$\mathbf{x} = (\mathbf{x}^1, \mathbf{x}^2, \dots, \mathbf{x}^N)^\top \in \mathcal{X} \subseteq \mathbb{R}^{d_x}, \tag{1}$$

where $\mathbf{x}^i \in \mathcal{X}^i \subseteq \mathbb{R}^{d_x^i}$ is the state of the ith robot, and $\mathcal{X}$ is the stacked state space. Similarly, the stacked action vector is defined as

$$\mathbf{u} = (\mathbf{u}^1, \mathbf{u}^2, \dots, \mathbf{u}^N)^\top \in \mathcal{U} \subseteq \mathbb{R}^{d_u}, \tag{2}$$

where $\mathbf{u}^i \in \mathcal{U}^i \subseteq \mathbb{R}^{d_u^i}$ is the controls applied to the ith robot. The dynamics of the stacked system are governed by

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}). \tag{3}$$

To employ gradient-based optimization, we assume that the Jacobian of $\mathbf{f}$ with respect to $\mathbf{x}$ and $\mathbf{u}$ is available, as is typical for motion planning in actuated multi-robot systems. Let $\mathbf{X} = \langle \mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T \rangle$ and $\mathbf{U} = \langle \mathbf{u}_0, \mathbf{u}_1, \dots, \mathbf{u}_{T-1} \rangle$ be a sequence of states and controls sampled at time $0, \Delta t, \dots, T\Delta t$ respectively, where Δt is a small timestep and the controls are constant during this timestep. We denote the start state as $\mathbf{x}_s$, the goal state as $\mathbf{x}_f$, and the collision-free state space as $\mathcal{X}_{\text{free}} \subset \mathcal{X}$, which accounts for robots as well as collisions against the environment. Our goal is to find a solution of states and actions from a start to a goal state in the minimal time T, which can be framed as the following optimization problem

$$\min_{\mathbf{X},\mathbf{U},T} \quad J(\mathbf{X}, \mathbf{U}, T),$$
$$\text{s.t.} \begin{cases} \mathbf{x}_{k+1} = \text{step}(\mathbf{x}_k, \mathbf{u}_k) \quad \forall k \in \{0, \dots, T-1\}, \\ \mathbf{u}_k \in \mathcal{U} \quad \forall k \in \{0, \dots, T-1\}, \\ \mathbf{x}_0 = \mathbf{x}_s, \quad \mathbf{x}_T = \mathbf{x}_f, \\ \mathbf{x}_k \in \mathcal{X}_{\text{free}} \subset \mathcal{X} \quad \forall k \in \{0, \dots, T\}, \\ \mathbf{g}(\mathbf{x}) = 0, \end{cases} \tag{4}$$

where the cost function is T and other task objectives (e.g., energy). The first constraint is the time-discretized system dynamics and the second constraint bounds actions to the admissible space $\mathcal{U}$. The third set of constraints enforces the given start and goal states. The final constraint $\mathbf{g}(\mathbf{x})$, assumed to be continuously differentiable (similar to [23]), enforces the physical rigidity coupling between robots, ensuring their motions satisfy the kinematic and dynamic interactions. We assume that the constraint set $\{\mathbf{x} \in \mathcal{X} \mid \mathbf{g}(\mathbf{x}) = 0\}$ admits a local mapping $\Phi : \mathcal{X}_m \to \mathcal{X}$ with minimal coordinates $\mathbf{x}_m \in \mathcal{X}_m$. To demonstrate the generality of the framework across embodiments, we present two robot platforms: (i) Unicycles, low-dimensional non-holonomic robots, rigidly connected in a "line"-formation. (ii) Multirotors transporting payloads with cables (modeled as rigid rods), high-dimensional underactuated robots. These examples are illustrative only. Our framework applies to any rigidly-coupled robot teams.

A. Unicycles With Rigid Rods (UR)

Consider a team of N unicycles connected by $N-1$ rigid rods of fixed lengths in a line formation. The state of the ith unicycle is described as $\mathbf{x}^i = (p_x^i, p_y^i, \theta^i)^\top$, where $\mathbf{p}^i = (p_x^i, p_y^i)^\top$

denote the position of the ith unicycle, and θ^i represents its orientation. The kinematic model for the unicycle is given by $\dot{\mathbf{x}}^i = \mathbf{C}^i\mathbf{u}^i$ as

$$\dot{\mathbf{x}}^i = \begin{pmatrix} \cos(\theta^i) & 0 \\ \sin(\theta^i) & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} v^i \\ \omega^i \end{pmatrix}, \tag{5}$$

where $\mathbf{u}^i = (v^i, \omega^i)^\top$ represents the linear and angular velocities, respectively. Let the length of the rod connecting the ith and $(i+1)^{\text{th}}$ unicycles be l^i. Then the coupling constraint $\mathbf{g}(\mathbf{x}) = 0$ is

$$\|\mathbf{p}^i - \mathbf{p}^{i+1}\| - l^i = 0, \quad i = 1, \ldots, N-1. \tag{6}$$

B. Multirotors Transporting a Payload (MP)

Consider a team of N multirotors transporting a cable-suspended payload. The ith multirotor is modeled as a rigid floating base with state $\mathbf{x}^i = (\mathbf{p}^i, \mathbf{R}^i, \mathbf{v}^i, \mathbf{\Omega}^i)^\top$. Here, $\mathbf{p}^i, \mathbf{v}^i \in \mathbb{R}^3$ represent the position and velocity in the world frame, $\mathbf{R}^i \in SO(3)$ represents the rotation matrix, and $\mathbf{\Omega}^i \in \mathbb{R}^3$ is the angular velocity expressed in the body frame. The action $\mathbf{u}^i \in \mathbb{R}^4$ is defined as the forces at the rotors, $\mathbf{u}^i = (f_1^i, f_2^i, f_3^i, f_4^i)^\top$. The dynamics are derived from Newton-Euler equations for rigid bodies as

$$\begin{aligned} \dot{\mathbf{p}}^i &= \mathbf{v}^i, & m^i\dot{\mathbf{v}}^i &= \mathbf{R}^i f_T^i \mathbf{e}_3 - m^i g \mathbf{e}_3, \\ \dot{\mathbf{R}}^i &= \mathbf{R}^i \hat{\mathbf{\Omega}}^i, & \mathbf{J}^i\dot{\mathbf{\Omega}}^i &= \mathbf{J}^i\mathbf{\Omega}^i \times \mathbf{\Omega}^i + \mathbf{M}^i, \end{aligned} \tag{7}$$

where m^i is the mass, $\mathbf{J}^i$ is the inertia matrix, g is the gravitational acceleration constant, $\mathbf{e}_3 = (0, 0, 1)^\top$. The $(\hat{\cdot})$ denotes the skew-symmetric mapping $\mathbb{R}^3 \to \mathfrak{so}(3)$.

The collective thrust and torques, $\boldsymbol{\eta}^i = (f_T^i, \mathbf{M}^i)^\top$, are linearly related to the motor forces $\mathbf{u}^i$ via a fixed and known actuation matrix.

Each multirotor is connected with a cable modeled as a rigid rod of length l^i to a point mass payload at $\mathbf{p}^0$ with mass m^0. Then the coupling constraint $\mathbf{g}(\mathbf{x}) = 0$ is

$$\|\mathbf{p}^0 - \mathbf{p}^i\| - l^i = 0, \quad i = 1, \ldots N. \tag{8}$$

IV. Approach

A. Pc-Dbcbs

Our motion planning approach, pc-dbCBS, extends and adapts db-CBS to the high-dimensional physically-coupled systems that are subject to physical constraints between the robots. Building upon this foundation, pc-dbCBS utilizes the three-level iterative framework of db-CBS by integrating additional definitions of conflicts and conversions of state representations to manage physical coupling constraints. In principle, pc-dbCBS utilizes the efficiency of the discrete search with the stacked state space, allowing the usage of pre-computed motion primitives for the single robot, along with the effectiveness of trajectory optimization on a minimal representation of the coupled system. The pseudocode in Algorithm 1 highlights major changes compared to db-CBS.

1) Single Robot Planning: The *first level* uses db-A* [24] to plan for each robot a trajectory with state discontinuous up to δ from a graph constructed of precomputed motion primitives $\mathcal{M}$. A motion primitive $\langle \mathbf{X}^i, \mathbf{U}^i, K \rangle$ is a sequence of states and actions satisfying the robot dynamics. For each robot, db-A* outputs sequences, adhering to (5) (unicycles) or (7) (multirotors). These sequences form *δ-discontinuity-bounded* solutions under the condition

$$\begin{aligned} & d(\mathbf{x}_{k+1}^i, \text{step}(\mathbf{x}_k^i, \mathbf{u}_k^i)) \leq \delta \quad \forall k, \\ & \mathbf{u}_k^i \in \mathcal{U}^i, \quad \mathbf{x}_k^i \in \mathcal{X}^i, \\ & d(\mathbf{x}_0^i, \mathbf{x}_s^i) \leq \delta, \quad d(\mathbf{x}_K^i, \mathbf{x}_f^i) \leq \delta, \end{aligned} \tag{9}$$

where d is a metric $d : \mathcal{X}^i \times \mathcal{X}^i \to \mathbb{R}$, which measures the distance between two states. In this work, we pre-compute each robot's set of motion primitives offline as in db-CBS, allowing heterogeneous robots through motion primitives tailored to each robot's dynamics and geometry.

Algorithm 1: pc-dbCBS: Changes to [6] are Highlighted.

```
1  while ¬TerminationCriteria () do
2      δ, M ← UpdateHyperParameters()
3      X_j, U_j ← HLDiscreteSearch(P, δ, M)
4      X_d, U_d ← Convert(X_j, U_j)
5      X, U ← Optimization(P, X_d, U_d)
6  def HLDiscreteSearch(P, δ, M):
       /* Initialize priority queue O by planning for individual robots
          independently                                                  */
7      while |O| > 0 do
8          P ← PriorityQueuePop(O)                    ▷ Lowest solution cost
9          if ¬ ResolveRobotCollision (P, O) then
10             if ¬ ResolvePhysicalConstraint (P, O) then
11                 if ¬ ResolvePCCollision (P, O) then
12                     return stacked solution
13 def ResolvePhysicalConstraint(P, O):
14     for k ∈ [1, K] do
15         C ← CheckForConflict(P.solution, k)
16         if C ≠ ∅ then
17             ExtractRandomConstraintAndAddToOpt (C, O)
18             return true
```

2) Conflict Resolution: The *second level* employs a search to analyze the motions generated by the first level, identifying conflicts between robots, and creating constraints that are then iteratively resolved by the first level. Conflicts are resolved through a hierarchical process that sequentially addresses three types of conflicts: inter-robot collisions, physical coupling violations, and coupling elements (e.g., rods) and environment collisions. These conflicts are resolved in Lines 9 to 11. Each type of conflict is resolved up to the tolerance δ over the full time horizon of the planned trajectory, before the next conflict type is considered.

The first level resolves robot-robot collisions by using the `ResolveRobotCollision` function (Line 9) to detect overlaps in planned motions. When conflicts are found, constraints are added to the priority queue $\mathcal{O}$, and the `HLDiscreteSearch` process iterates with these updates. This behavior is identical to db-CBS.

The second level resolves the physical coupling constraints. The `ResolvePhysicalConstraint` function (Line 10) evaluates the solution of the robots' formations up to given bounds. For each step k, the constraint $\|\mathbf{g}(\mathbf{x})\| < \delta$ is checked.

If a violation is detected, only a single new node is added to $\mathcal{O}$, constraining a randomly picked single robot motion. Note that in the traditional CBS framework N nodes would need to be added, as it is unknown which combination of robots is causing the constraint violation. Picking a single robot instead reduces the size of $\mathcal{O}$, but has some theoretical drawbacks, as discussed later.

The final level of conflict resolution involves collision checking between the physical coupling elements (e.g., rods) and the obstacles in the environment. The `ResolvePCCollision` function (Line 11) constructs artificial collision shapes for the coupling elements and evaluates whether they collide with obstacles. If a collision is identified, one additional node is added to $\mathcal{O}$, constraining the affected robot. In principle, inter-cable collisions could be defined as conflicts; however, this would significantly increase computational overhead as they are already addressed during the optimization step as collision shapes (e.g., rods or cylinders), we find this unnecessary in practice.

If no conflicts are detected at all levels, the algorithm returns a stacked space solution that adheres to all robot-robot, physical coupling, and obstacle collisions constraints up to the bounded tolerance δ.

3) Stacked to Constrained Systems: The next step involves converting the stacked space solution from the discrete search to the constrained physically-coupled dynamical system (Line 4). This is achieved by representing the state into a minimal representation $\mathbf{x}_m \in \mathcal{X}_m$. The dynamics of the physically coupled system can be projected on the coupling constraints $\mathbf{g}(\mathbf{x})$ in (4) using the minimal representation, which are governed by $\dot{\mathbf{x}}_m = \mathbf{f}_m(\mathbf{x}_m, \mathbf{u})$.

4) Trajectory Optimization: After the discrete stacked state solution is mapped to the minimal state representation, we use trajectory optimization to refine the solution from the discrete search and to repair all the discontinuities of δ. The optimization problem is reformulated using the minimal state representation as

$$\begin{aligned}
&\min_{\mathbf{X},\mathbf{U},\Delta t} \quad \sum_k (\Delta t - \Delta t_0)^2 + \beta_1 \|\mathbf{u}_k\|^2 \\
&+ \beta_2 \|\ddot{\boldsymbol{x}}_m(\mathbf{x}_{m_k}, \mathbf{u}_k)\|^2 \\
&\text{s.t.} \begin{cases} \mathbf{x}_{m_{k+1}} = \text{step}(\mathbf{x}_{m_k}, \mathbf{u}_k) \quad \forall k \in \{0, \dots, T-1\}, \\ \mathbf{u}_k \in \mathcal{U} \quad \forall k \in \{0, \dots, T-1\}, \\ \mathbf{x}_{m_0} = \mathbf{x}_{m_s}, \quad \mathbf{x}_{m_T} = \mathbf{x}_{m_f}, \\ \mathbf{x}_{m_k} \in \mathcal{X}_{m_{\text{free}}} \subset \mathcal{X}_m \quad \forall k \in \{0, \dots, T\}. \end{cases}
\end{aligned} \tag{10}$$

Here, the cost function minimizes the deviation of the time step Δt from a nominal value Δt_0, penalizes the control effort $\|\mathbf{u}_k\|^2$, and reduces the system's dynamic accelerations $\|\ddot{\boldsymbol{x}}(\mathbf{x}_k, \mathbf{u}_k)\|^2$ to improve trajectory smoothness. The weighting parameters β_1 and β_2 are used to balance the contributions of the control effort and dynamic smoothness terms. This part is identical to prior formulations such as [5].

5) Anytime Planning: Our method iteratively refines solutions by updating two parameters (Line 2). First, δ is gradually decreased with a predefined rate. Second, solutions from the optimization step, including failed ones, are extracted, split, and transformed from the minimal representation to single-robot motion primitives. Then these are added to the motion primitive database $\mathcal{M}$ alongside newly sampled primitives with a predefined rate.

B. Case Study: Unicycles With Rigid Rods (UR)

1) Constrained Dynamics: The system can be minimally represented by the state vector

$$\mathbf{x}_m = (p_x^1, p_y^1, \theta^1, \dots, \theta^N, \alpha^1, \dots, \alpha^{N-1})^\top, \tag{11}$$

where α^i represents the orientation of the ith rod. We have $\mathbf{x}_m \in \mathbb{R}^{2N} \times (SO(2))^{2N-1}$, and the position of the $(i+1)^{\text{th}}$ unicycle is expressed by

$$p_x^{i+1} = p_x^i + l^i \cos(\alpha^i), \quad p_y^{i+1} = p_y^i + l^i \sin(\alpha^i). \tag{12}$$

The kinematics of the N unicycles with rigid rods are governed by the constraints imposed by the distance between the connected unicycles. Let the relative position and its derivative with respect to time of two neighboring robots i and j be

$$\begin{aligned} dx_{ij} &= p_x^j - p_x^i, & dy_{ij} &= p_y^j - p_y^i \\ \dot{dx}_{ij} &= \dot{p}_x^j - \dot{p}_x^i, & \dot{dy}_i &= \dot{p}_y^j - \dot{p}_y^i \end{aligned} \tag{13}$$

Thus, the constraint enforcing a fixed distance between the ith and j^{th} consecutive unicycles is given by

$$dx_{ij}^2 + dy_{ij}^2 = {l^i}^2. \tag{14}$$

Differentiating (14) with respect to time yields

$$2dx_{ij}\dot{dx}_{ij} + 2dy_{ij}\dot{dy}_{ij} = 0, \tag{15}$$

which ensures that the velocities of the unicycles are consistent with the physical constraints imposed by the rods. To incorporate these constraints into the system kinematics, we construct the Jacobian matrix $\mathbf{A}$ with dimensions $(n-1) \times 3n$. The rod constraints in (15) can then be expressed compactly as

$$\mathbf{A}\dot{\mathbf{x}}^{ur} = 0, \tag{16}$$

where $\mathbf{x}^{ur} = (p_x^1, p_y^1, \theta^1, \dots, p_x^N, p_y^N, \theta^N)$. This formulation ensures that the system dynamics remain consistent with the physical constraints imposed by the rods.

Inspired by [23], the kinematics are then projected into the constraint-consistent space using a projection matrix $\mathbf{G} \in \mathbb{R}^{3n \times 2n}$

$$\dot{\mathbf{x}}^{ur} = \mathbf{G}\mathbf{u}, \quad \mathbf{G} = \mathbf{B} - \mathbf{A}^\dagger(\mathbf{A}\mathbf{B}), \tag{17}$$

with $\mathbf{A}^\dagger = \mathbf{A}^\top(\mathbf{A}\mathbf{A}^\top)^{-1}$. The input mapping matrix for the unicycles $\mathbf{B} \in \mathbb{R}^{2n \times 2n}$ is a block-diagonal matrix defined as $\mathbf{B} = \text{diag}(\mathbf{C}^1, \mathbf{C}^2, \dots, \mathbf{C}^N)$, where each $\mathbf{C}^i$ is defined in (5).

The angle of each rod α^i and its angular velocity $\dot{\alpha}^i$ are computed by

$$\begin{aligned} \alpha^i &= \arctan(dy_{ij}, dx_{ij}) \\ \dot{\alpha}^i &= \frac{1}{{l^i}^2} dx_{ij}\dot{dy}_{ij} - dy_{ij}\dot{dx}_{ij} \quad , i \in \{1, \dots, N\}. \end{aligned} \tag{18}$$

The dynamics $\mathbf{f}_m$ is then computed using (17), (18) and the action vector $\mathbf{u}$.

2) Conflicts: The conflict in the second level arises from maintaining a certain distance decided by the length of the rigid rod. Consider $l^i_{c_k} = \sqrt{dx^2_{ij,k} + dy^2_{ij,k}} \quad \forall (i, i+1)$ to be the actual relative distance between each consecutive pair of unicycles at the kth step. Then, a conflict for the robot pair $(i, i+1)$ occurs if

$$|l^i_{c_k} - l^i| > \delta, \tag{19}$$

where l^i is the nominal rod length, and δ defines the allowable tolerance.

C. Case Study: Multirotors Transporting a Payload (MP)

1) Constrained Dynamics: The dynamics of multirotors with cable-suspended payloads, as presented in [5], are described using the minimal state representation

$$\mathbf{x}_m = (\mathbf{p}^0, \dot{\mathbf{p}}^0, \mathbf{s}^1, \boldsymbol{\omega}^1, \mathbf{R}^1, \boldsymbol{\Omega}^1, \ldots, \mathbf{s}^N, \boldsymbol{\omega}^N, \mathbf{R}^N, \boldsymbol{\Omega}^N)^\top, \tag{20}$$

where $\mathbf{p}^0 \in \mathbb{R}^3$ is the payload's position, $\dot{\mathbf{p}}^0 \in \mathbb{R}^3$ is the payload velocity, $\mathbf{s}^i \in \mathbb{S}^2$ are the cable unit vectors pointing from the UAV to the payload (with $\mathbb{S}^2 = \{\mathbf{s} \in \mathbb{R}^3 \big| \|\mathbf{s}\| = 1\}$), and $\boldsymbol{\omega}^i \in \mathbb{R}^3$ are the cable angular velocities, where $i \in \{1, \ldots, N\}$.

The UAV position and velocity vectors are $\mathbf{p}^i \in \mathbb{R}^3$ and $\dot{\mathbf{p}}^i \in \mathbb{R}^3$ are computed as

$$\mathbf{p}^i = \mathbf{p}^0 - l^i \mathbf{s}^i, \quad \dot{\mathbf{p}}^i = \dot{\mathbf{p}}^0 - l^i \dot{\mathbf{s}}^i. \tag{21}$$

The dynamics $\mathbf{f}_m$ of the system is defined as

$$\begin{aligned}
&\dot{\mathbf{s}}^i = \boldsymbol{\omega}^i \times \mathbf{s}^i, \quad \mathbf{s}^i = \frac{\mathbf{p}^0 - \mathbf{p}^i}{\|\mathbf{p}^0 - \mathbf{p}^i\|}, \\
&\mathbf{M_t}(\ddot{\mathbf{p}}^0 + g\mathbf{e}_3) = \sum_{i=1}^{n} (f^i_T \mathbf{R}^i \mathbf{e}_3 - m^i l^i \|\boldsymbol{\omega}^i\|^2 \mathbf{s}^i), \\
&m^i l^i \dot{\boldsymbol{\omega}}^i = m^i \hat{\mathbf{s}}^i (\ddot{\mathbf{p}}^0 + g\mathbf{e}_3) - f^i_T \hat{\mathbf{s}}^i \mathbf{R}^i \mathbf{e}_3, \\
&\dot{\mathbf{R}}^i = \mathbf{R}^i \hat{\boldsymbol{\Omega}}^i, \quad \mathbf{J}\dot{\boldsymbol{\Omega}}^i = \mathbf{J}\boldsymbol{\Omega}^i \times \boldsymbol{\Omega}^i + \mathbf{M}^i,
\end{aligned} \tag{22}$$

where $\ddot{\mathbf{p}}^0$ is the payload acceleration, $\mathbf{M}_t = m^0 \mathbf{I}_3 + \sum_{i=1}^{n} m^i \mathbf{s}^i \mathbf{s}^{i\top}$ and g is the gravitational acceleration.

2) Conflicts: A challenge for the conflict detection is that the constraint $\mathbf{g}(\mathbf{x})$ depends on the payload position $\mathbf{p}^0$, which is not part of the stacked state. Thus, we estimate the payload positions $\mathbf{p}^{0_e}_k$ by solving the following optimization problem

$$\begin{aligned}
\min_{\mathbf{p}^{0_e}_k} \sum_{i=1}^{N} &(\|\mathbf{p}^{0_e}_k - \mathbf{p}^i_k\| - l^i)^2 + \mu \|\mathbf{p}^{0_e}_k - \mathbf{p}^{0_d}_k\| \\
&+ \lambda (\min_i \{p^i_{z_k}\} - p^{0_e}_{z_k} - l_{\min})^2,
\end{aligned} \tag{23}$$

where μ and λ are weighting parameters, $\mathbf{p}^{0_d}_k$ is used to penalize jumps of the payload estimate from the previous solution, and $l_{\min}$ is the cable length associated with the robot closest to the payload $\min_i \{p^i_{z_k}\}$ in the z-axis among all robots. This cost function ensures that the payload position minimizes deviations from nominal cable lengths, penalizes significant changes in the payload position, and guarantees the estimated payload position remains below the robots' positions.

Once the payload position $\mathbf{p}^{0_e}_k$ is estimated, the actual cable lengths are computed as

$$l^i_{c_k} = \|\mathbf{p}^{0_e}_k - \mathbf{p}^i_k\|, \quad \forall i \in \{1, \ldots, N\}. \tag{24}$$

A conflict in the second level for all robots arises if any cable length $l^i_{c_k}$ deviates from the nominal cable length l^i by more than the tolerance δ

$$|l^i_{c_k} - l^i| > \delta. \tag{25}$$

Note that in this case all robots participate in the conflict.

D. Theoretical Remarks

pc-dbCBS inherits the probabilistic completeness and asymptotic optimality of db-CBS [6, Theorem 1]. In particular [6, Theorem 1] states that db-CBS is asymptotically optimal (implying probabilistic completeness), i.e.,

$$\lim_{n \to \infty} P(c_n - c^* > \varepsilon) = 0 \quad \forall \varepsilon > 0, \tag{26}$$

where c_n is the cost in iteration n and c^* is the optimal cost. In db-CBS, at each iteration n, the discrete search finds the optimal solution within δ, if one exists, yielding the optimal discrete cost c_n. At each iteration the motion primitive library grows and δ shrinks, expanding the discrete search graph. For pc-dbCBS, the same argument of the proof as in [6, Theorem 1] still holds with the highlighted changes in Algorithm 1, because the outer-loop (Lines 1 to 5) is unchanged.

We note that the inner loop Lines 6 to 12 of our proposed algorithm violates some key assumptions of CBS: i) for completeness, all possible alternatives need to be considered in the open list (which we violate in `ResolvePhysicalConstraints` by only including a randomly-picked single new entry); and ii) for optimality, we need to resolve conflicts in the order of their occurrence in time (which we violate using our hierarchical approach that checks for a certain type of conflict over the whole time horizon). These changes might result producing near-optimal results of a single inner loop iteration; however the discrete search does not deterministically prune potential solution trajectories, thus the key properties of asymptotic optimality and probabilistic completeness are retained. However, our choice impacts the runtime as potentially more outer loop iterations are needed.

V. Results

To validate the performance of our method, we compare with a state-of-the-art baseline method [5], in both simulation and real experiments. We evaluate both methods on two systems: unicycles connected by rigid rods and multirotors transporting payloads with cables. To this end, we extend the baseline to the unicycle case with rigid rods.

The baseline relies on OMPL [25] and for optimization on Dynoplan [24], a motion planning framework built on Crocoddyl [26]. We extend the implementation of Dynoplan to include the unicycles with rods, implemented in C++. Both kinodynamic

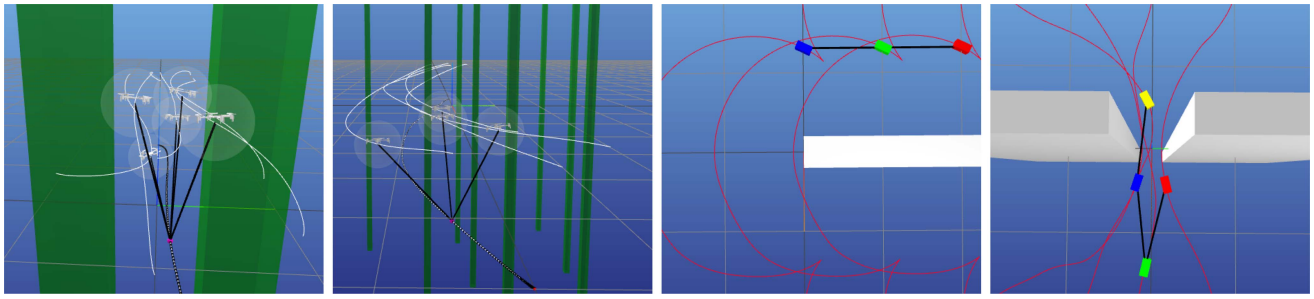

Fig. 2. Simulation environments from left to right: window (5 multirotors), forest (4 multirotors), wall (3 unicycles), window (4 unicycles). Note that the forest environment is the same for both robot types.

TABLE I
SIMULATION OF MULTIROTORS TRANSPORTING A PAYLOAD (MP). SHOWN ARE MEAN VALUES FOR THE SUCCESS RATE, COST AND COMPUTATIONAL TIME OVER 40 RUNS WITH A TIMELIMIT OF 350 s. STANDARD DEVIATION IS SMALL GRAY. PERCENTAGES ARE SUCCESS RATES. F: FAILED.

Environment, robots	Success [%] ↑				Cost [s] ↓				Time [s] ↓			
	UR		MP		UR		MP		UR		MP	
	Ours	BL	Ours	BL	Ours	BL	Ours	BL	Ours	BL	Ours	BL
Window, 2 robots	**100.0**	**100.0**	**100.0**	92.5	**4.7** 0.0	10.3 0.0	**2.0** 0.1	6.3 1.3	**0.4** 0.1	350.8 0.2	**21.8** 11.0	376.8 12.8
Window, 3 robots	**100.0**	**100.0**	**97.5**	65.0	**6.1** 0.2	17.4 2.9	**2.0** 0.3	6.5 1.9	**1.5** 1.1	351.9 0.7	**53.6** 32.8	402.8 21.4
Window, 4 robots	**100.0**	75.0	**77.5**	10.0	**8.6** 0.4	13.7 3.6	**2.1** 0.3	6.4 2.1	**3.3** 0.6	352.9 0.9	**103.5** 43.1	461.0 58.6
Window, 5 robots	**95.0**	62.5	**30.0**	0.0	**8.7** 0.3	12.4 1.8	**2.1** 0.3	F	**17.8** 27.7	354.1 1.5	**220.1** 55.5	F
Window, 6 robots	**90.0**	27.5	**12.5**	0.0	**11.5** 0.4	18.8 3.1	**2.4** 0.2	F	**31.0** 36.1	356.7 1.8	**226.1** 32.0	F
Forest, 2 robots	**100.0**	75.0	**100.0**	**100.0**	**9.2** 0.1	12.7 2.1	**2.4** 0.2	6.8 1.3	**1.0** 0.4	350.7 0.2	**22.9** 9.5	378.5 10.8
Forest, 3 robots	**100.0**	72.5	82.5	**92.5**	**11.8** 0.2	12.4 1.0	**2.8** 0.2	7.1 1.6	**3.5** 1.6	350.9 0.2	**53.4** 31.7	389.9 16.0
Forest, 4 robots	**97.5**	12.5	45.0	**92.5**	**12.6** 0.5	13.6 1.5	**3.1** 0.4	7.1 1.8	**12.4** 23.4	352.0 1.1	**65.3** 25.7	417.1 31.6
Forest, 5 robots	**87.5**	12.5	15.0	**90.0**	**16.1** 0.7	18.0 4.0	**3.3** 0.4	7.8 1.3	**28.2** 50.8	356.2 3.0	**99.5** 22.2	606.4 81.9
Forest, 6 robots	**47.5**	7.5	10.0	**65.0**	**18.6** 1.1	27.0 2.0	**3.2** 0.3	8.0 1.3	**44.2** 46.8	358.5 1.5	**117.8** 8.7	667.5 90.5

motion planners use the Flexible Collision Library (FCL) [27] for collision checking. For solving (23), we rely on NLopt.

A. Simulations

For multirotors with cable-suspended payload, we validate our method in simulation using a software-in-the-loop setup. This setup executes the actual flight controller code [28] designed for Bitcraze Crazyflie 2.1 multirotors. For unicycles, we implemented a nonlinear controller [29] in Python. In simulation, as shown in Fig. 2, we test pc-dbCBS (**Ours**) and the baseline (**BL**) on three distinct scenarios for both systems (see supplemental video). For n-unicycles with rods and multirotors with cable-suspended payloads, the first scenario involves a window environment where the goal is to pass through a narrow passage. The second scenario is a forest-like environment with dense obstacles. To evaluate the completeness of both methods, we designed a third scenario specifically for unicycles with rods: we constrain the angular velocity of each unicycle in a wall environment, allowing only clockwise rotations ($\omega^i \in [0, 0.5]\ \mathrm{rad/s}$). For each scenario, we gradually increase the number of robots from two to six. We denote (F) as failed attempts. All experiments were conducted on a workstation (AMD Ryzen Threadripper PRO 5975WX @ 3.6 GHz, 128 GB RAM, Ubuntu 22.04). Each experiment was repeated 40 times, with a runtime limit for both planners of 350 s.

1) Comparison Metrics: To evaluate the performance of our method, we consider three key metrics on 25 problem instances: success rate, cost, and computation time. The results are summarized in Tables I and II. The success rate is defined as the proportion of problem instances where a feasible trajectory is successfully computed by the motion planner. The cost metric is the total time of the trajectory from the optimization process for both methods, as both utilize the same trajectory optimization step. Finally, the computation time is the motion planner's runtime required to compute a feasible trajectory.

TABLE II
SIMULATION OF UNICYCLES WITH RODS (UR). SEE TABLE I.

Environment, robots	Success [%] ↑		Cost [s] ↓		Time [s] ↓	
	Ours	BL	Ours	BL	Ours	BL
Wall, 2 robots	**97.5**	F	**13.9** 1.0	F	**2.6** 2.2	F
Wall, 3 robots	**70.0**	F	**16.1** 1.3	F	**9.4** 9.6	F
Wall, 4 robots	**65.0**	F	**15.7** 1.1	F	**12.6** 8.5	F
Wall, 5 robots	**62.5**	F	**16.4** 1.3	F	**16.1** 10.7	F
Wall, 6 robots	**50.0**	F	**16.5** 0.9	F	**20.7** 7.5	F

2) Results: For multirotors with cable-suspended payloads tested in window and forest environments, pc-dbCBS consistently generates 60% lower cost (i.e., faster trajectories) than the baseline. **Ours** achieves a higher success rate in narrow-passage scenarios (e.g., window environment).

For unicycles connected by rods in dense environments, pc-dbCBS outperforms the baseline in both success rate and cost. However, in dense scenes like the forest for multirotors with cable-suspended payloads, the success rate of pc-dbCBS decreases as the team grows, due to the combinatorial explosion in cable-obstacles collision conflicts, as the finite set of motion primitives cannot explore every branch in the graph. These

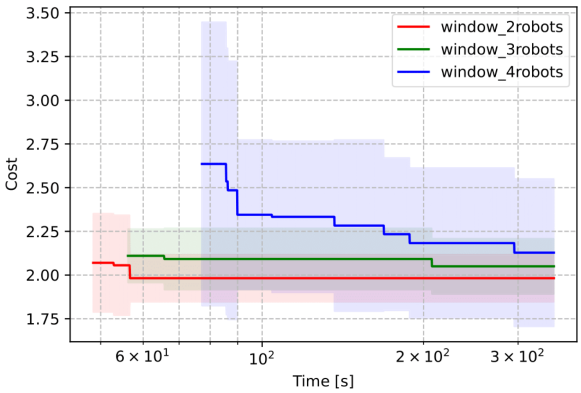

Fig. 3. Anytime planning of pc-dbCBS for three different example scenarios for multirotors with payload (MP).

failures arise from practical limits on time and the number of primitives, thus, this does not contradict pc-dbCBS retaining its probabilistic completeness. Furthermore, pc-dbCBS demonstrates in Table II high success rates in the wall environment, where unicycles are restricted to clockwise rotation. In contrast, the baseline fails to generate any feasible trajectories in this scenario, because it does not reason about dynamic limits, causing the optimization step to consistently fail. In terms of computation time, the geometric planner in the baseline executes until it times out, followed by additional time for the optimization step. In contrast, pc-dbCBS applies the anytime planning property across the entire framework, allowing us to report both the cost and computation time of the first valid solution, highlighting its efficiency in finding solutions fast with better cost.

We study the effect of the maximum number of available motion primitives in UR Window (4 robots, 40 trials), testing limits of 25, 100, 200, 500, 1000, and 2000 primitives, and observe success rates of $0\,\%$, $37.5\,\%$, $50\,\%$, $60\,\%$, $75\,\%$, and $100\,\%$, respectively. We ablate on conflict levels in MP Forest (4 robots, 40 trials), observing success rates of $7.5\,\%$ for level 1 (robot collisions only), $12.5\,\%$ for levels 1+2 (robot collisions and cable-length tolerance), $32.5\,\%$ for levels 1+3 (robot collisions and cable/environment), and $45\,\%$ for the full tri-level strategy, confirming the importance of all levels.

Moreover, the supplemental material demonstrates that our approach handles heterogeneous robots (e.g., 2 unicycles with a double integrator connected with rigid rods).

3) Optimality and Anytime Planning: In our work, we use the term anytime synonymously with asymptotic optimality, which is typical in the motion planning literature. The formal properties of *anytime* [30], with the exception of the interruptibility before the first solution is found, are maintained. By decreasing δ and adding more motion primitives to the graph search step (Section IV-A5), we achieve a consistent cost reduction at each iteration, thereby demonstrating the method's asymptotic optimality as shown in Fig. 3 for two, three, and four multirotors with payloads in the window environment. Furthermore, the optimization process operates in an anytime manner, allowing it to improve the solution over time and stops when the predefined time limit is reached. This ensures that the method can effectively balance solution quality and computational efficiency.

TABLE III
PHYSICAL EXPERIMENTS WITH MULTIROTORS WITH PAYLOAD (MP). ENERGY, TRACKING ERROR, AND TRAJECTORY COST OVER 10 TRIALS.

Environment	Energy [Wh] ↓		Error [m] ↓		Time [s] ↓	
	Ours	BL	Ours	BL	Ours	BL
Window, 2 robots	**0.006** 0.00	0.01 0.00	0.08 0.05	**0.05** 0.03	**4.3**	7.5
Window, 3 robots	**0.007** 0.00	0.02 0.00	0.15 0.06	**0.08** 0.04	**4.2**	8.5
Forest, 2 robots	**0.007** 0.00	0.01 0.00	0.06 0.03	**0.03** 0.02	**5.0**	8.2
Forest, 3 robots	**0.009** 0.00	0.01 0.00	0.12 0.06	**0.07** 0.05	**4.5**	7.7

B. Physical Experiments

We conduct real-world experiments on both platforms: multirotors with cable-suspended payload and unicycles connected by rigid rods, see Fig. 1. For the multirotors with payload platform, we test scenarios with two and three multirotors. The experiments include two environments: a window-like environment requiring the robots to pass through a narrow passage and a forest-like environment with dense obstacles, similar to the scenarios that were tested for the baseline. For the unicycles with rigid rods platform, we conduct experiments with two and three robots in a wall-like environment. Here, the dynamic constraints on the angular velocities of the unicycles are $\omega^i \in [-0.5, 0.5]\ \mathrm{rad/s}$. Additionally, we demonstrate a practical use-case involving three unicycles functioning as garbage collectors, as shown in the supplemental material.

1) Multirotors With Payloads: The experiments utilize Bitcraze Crazyflie 2.1 (CF) multirotors, which are small ($9\,\mathrm{cm}$ rotor-to-rotor) and lightweight ($34\,\mathrm{g}$), and are commercially available. An existing flight controller [28] is run on-board the STM32-based flight controller (168 MHz, 192 kB RAM), which also handles an extended Kalman filter for state estimation. For all scenarios, we use the open-sourced baseline implementation [5]. On the host side, we used Crazyswarm2, an extension of Crazyswarm [31], which leverages ROS 2 [32] for commanding multiple CFs.

2) Unicycles With Rods: We use commercially off-the-shelf differential-drive robots of type Polulu 3pi+ 2040. The Cortex M0+ microcontroller runs MicroPython with a nonlinear controller [29]. For state estimation, we equip the robots with low-latency radios (nRF52840) and broadcast the motion capture pose at 50 Hz. Robots are physically connected with 3D-printed rigid rods using revolute joints.

3) Results: We successfully execute the generated trajectories for both systems across all environments over 10 trials, except for the baseline in the three-unicycle scenario, where no successful attempts were recorded, see Table III. The trajectory times of pc-dbCBS are on average 50% faster than the baseline and were successfully tracked by the existing controllers. For the cable-suspended system, the executed trajectories demonstrate that our method on average consumes 50% less energy than the baseline. As expected, the average tracking error of pc-dbCBS for the multirotors with payloads degrades at the higher execution speed due to system uncertainties, such as model mismatches and state estimation inaccuracies. However, this does not impact the success rate of the experiments. For the unicycles, the average tracking error for pc-dbCBS three robots is $0.12\,\mathrm{m}$ executed in $8.9\,\mathrm{s}$. Similarly for two robots,

the average tracking error is $0.12\,\mathrm{m}$ executed in $6.4\,\mathrm{s}$ and the baseline is $0.27\,\mathrm{m}$ in $13.2\,\mathrm{s}$. This is because the generated path from the baseline is overly curved, even though it is feasible. This suboptimality explains why the baseline results in collisions with obstacles for three robots.

VI. Conclusion

We present pc-dbCBS, a novel kinodynamic motion planner for physically-coupled robot teams. Algorithmically, our kinodynamic planner extends db-CBS for physically-coupled multi-robot systems. Our key insight is that it is beneficial to use different state representations in the same planning framework simultaneously. Specifically, we use a stacked state representation for a discrete search over motion primitives and a minimal state representation for optimization with differential dynamic programming. Our approach, pc-dbCBS, achieves up to 60% lower cost than the baseline, and energy consumption is reduced to 10-40% in comparison to the baseline, depending on the number of robots and their type. In addition, we emphasize the importance of all conflict resolution levels and the necessity of state space representation switching for such high-dimensional problems.

In the future, the scalability of the proposed method needs to be improved further to handle larger teams in complex environments. Moreover, a closer coupling of motion planning and controls is needed in order to plan agile trajectories that can be tracked robustly on real robots.

References

[1] K. Sreenath and V. Kumar, "Dynamics, control and planning for cooperative manipulation of payloads suspended by cables from multiple quadrotor robots," in *Proc. Robot., Sci. Syst.*, 2013.

[2] R. Tallamraju, D. H. Salunkhe, S. Rajappa, A. Ahmad, K. Karlapalem, and S. V. Shah, "Motion planning for multi-mobile-manipulator payload transport systems," in *Proc. IEEE 15th Int. Conf. Automat. Sci. Eng.*, 2019, pp. 1469–1474.

[3] S. Sun and A. Franchi, "Nonlinear MPC for full-pose manipulation of a cable-suspended load using multiple UAVs," in *IEEE Int. Conf. Unmanned Aircr. Syst.*, 2023, pp. 969–975.

[4] G. Li and G. Loianno, "Nonlinear model predictive control for cooperative transportation and manipulation of cable suspended payloads with multiple quadrotors," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2023, pp. 5034–5041.

[5] K. Wahba, J. Ortiz-Haro, M. Toussaint, and W. Hönig, "Kinodynamic motion planning for a team of multirotors transporting a cable-suspended payload in cluttered environments," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2024, pp. 12750–12757.

[6] A. Moldagalieva, J. Ortiz-Haro, M. Toussaint, and W. Hönig, "db-CBS: Discontinuity-bounded conflict-based search for multi-robot kinodynamic motion planning," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2024, pp. 14569–14 575.

[7] L. Rapetti, Y. Tirupachuri, A. Ranavolo, T. Kawakami, T. Yoshiike, and D. Pucci, "Shared control of robot-robot collaborative lifting with agent postural and force ergonomic optimization," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2021, pp. 9840–9847.

[8] A. Afifi, M. van Holland, and A. Franchi, "Toward physical human-robot interaction control with aerial manipulators: Compliance, redundancy resolution, and input limits," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2022, pp. 4855–4861.

[9] S. Bamert et al., "Geranos: A novel tilted-rotors aerial robot for the transportation of poles," *IEEE Robot. Automat. Mag.*, vol. 31, no. 2, pp. 66–77, Jun. 2024.

[10] M. Tognon et al., "A truly-redundant aerial manipulator system with application to push-and-slide inspection in industrial plants," *IEEE Trans. Robot. Automat. Lett.*, vol. 4, no. 2, pp. 1846–1851, Apr. 2019.

[11] R. Gayle, S. Redon, A. Sud, M. C. Lin, and D. Manocha, "Efficient motion planning of highly articulated chains using physics-based sampling," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2007, pp. 3319–3326.

[12] M. Kulkarni, H. Nguyen, and K. Alexis, "The reconfigurable aerial robotic chain: Shape and motion planning," *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 9295–9302, 2020.

[13] M. S. Saleem, R. Sood, S. Onodera, R. Arora, H. Kanazawa, and M. Likhachev, "Search-based path planning for a high dimensional manipulator in cluttered environments using optimization-based primitives," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2021, pp. 8301–8308.

[14] K. Hauser and Y. Zhou, "Asymptotically optimal planning by feasible kinodynamic planning in a state–cost space," *IEEE Trans. Robot.*, vol. 32, no. 6, pp. 1431–1443, Dec. 2016.

[15] D. Malyuta et al., "Convex optimization for trajectory generation: A tutorial on generating dynamically feasible trajectories reliably and efficiently," *IEEE Control Syst. Mag.*, vol. 42, no. 5, pp. 40–113, Oct. 2022.

[16] T. A. Howell, B. E. Jackson, and Z. Manchester, "ALTRO: A fast solver for constrained trajectory optimization," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 7674–7679.

[17] J. Kottinger, S. Almagor, and M. Lahijanian, "Conflict-based search for multi-robot motion planning with kinodynamic constraints," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2022, pp. 13494–13 499.

[18] J. Chen, J. Li, C. Fan, and B. C. Williams, "Scalable and safe multi-agent motion planning with nonlinear dynamics and bounded disturbances," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 11237–11245.

[19] M. Manubens, D. Devaurs, L. Ros, and J. Cortés, "Motion planning for 6-D manipulation with aerial towed-cable systems," in *Proc. Robot., Sci. Syst.*, 2013.

[20] H. G. d. Marina and E. Smeur, "Flexible collaborative transportation by a team of rotorcraft," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2019, pp. 1074–1080.

[21] C. Gabellieri and A. Franchi, "Differential flatness and manipulation of elasto-flexible cables carried by aerial robots in a possibly Viscous environment," in *Proc. IEEE Int. Conf. Unmanned Aircr. Syst.*, 2023, pp. 963–968.

[22] Y. Zhang, J. Xu, C. Zhao, and J. Dong, "IF-based trajectory planning and cooperative control for transportation system of cable suspended payload with multi UAVs," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2023, pp. 635–642.

[23] A. De Luca, G. Oriolo, and C. Samson, "Feedback control of a nonholonomic car-like robot," in *Robot Motion Plan. Control*, 1998, pp. 171–253.

[24] J. Ortiz-Haro, W. Hönig, V. N. Hartmann, and M. Toussaint, "iDb-A*: Iterative search and optimization for optimal kinodynamic motion planning," *IEEE Trans. Robot.*, vol. 41, pp. 2031–2049, 2025.

[25] I. A. Şucan, M. Moll, and L. E. Kavraki, "The open motion planning library," *IEEE Robot. Automat. Mag.*, vol. 19, no. 4, pp. 72–82, Dec. 2012.

[26] C. Mastalli et al., "Crocoddyl: An efficient and versatile framework for multi-contact optimal control," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2020, pp. 2536–2542.

[27] J. Pan, S. Chitta, and D. Manocha, "FCL: A general purpose library for collision and proximity queries," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2012, pp. 3859–3866.

[28] K. Wahba and W. Hönig, "Efficient optimization-based cable force allocation for geometric control of a multirotor team transporting a payload," *IEEE Trans. Robot. Automat. Lett.*, vol. 9, no. 4, pp. 3688–3695, Apr. 2024.

[29] Y. Kanayama, Y. Kimura, F. Miyazaki, and T. Noguchi, "A stable tracking control method for an autonomous mobile robot," in *Proc. IEEE Int. Conf. Robot. Automat.*, 1990, vol. 1, pp. 384–389.

[30] S. Zilberstein, "Using anytime algorithms in intelligent systems," *AI Mag.*, vol. 17, no. 3, pp. 73–73, 1996.

[31] J. A. Preiss, W. Honig, G. S. Sukhatme, and N. Ayanian, "Crazyswarm: A large nano-quadcopter swarm," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2017, pp. 3299–3304.

[32] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Sci. Robot.*, vol. 7, no. 66, 2022, Art. no. eabm6074.